

P0451r0

Future-Proofing Parallel Algorithms

Exception Handling

Published Proposal, 14 October 2016

This version:

<http://wg21.link/P0451r0>

Authors:

[Bryce Adelstein Lelbach](#) (Lawrence Berkeley National Laboratory)

[Alisdair Meredith](#) (Bloomberg)

Audience:

SG1, LEWG, LWG

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Abstract

Addressing US NB comments on the `terminate()`-on-uncaught-exception behavior introduced by [\[P0394r4\]](#) and ensuring future extensibility of parallel algorithm exception handling.

Table of Contents

- 1 **Background**
- 2 **Feedback on P0394r4**
- 3 **Proposed Resolutions**
- 4 **Acknowledgement**

References

Informative References

§ 1. Background

[\[P0394r4\]](#), which was adopted for C++17 at the 2016 Oulu meeting, simplified the exception handling behavior of parallel algorithms. Now, if the invocation of an element access function (operations on the iterators, operations on sequence elements, user-provided function objects and operations on those function objects) exits via uncaught exception during the execution of a parallel algorithm, `terminate()` is called. This change removes the previous inconsistency in exception handling behavior between execution policies. It also removed the need for `exception_lists`, which SG1 and LEWG felt had not received sufficient implementation experience and were underspecified, lacking a mechanism for users to modify and construct them.

A user-constructible and modifiable `exception_list` would need to be a concurrent data structure due to the possibility of concurrent access via ABI functions. The `exception_list` in the Parallelism TS v1 would have no such requirement. Design of a user-constructible and modifiable `exception_list` and a C++ library interface for the kind of concurrent data structure required to implement it (a persistent container) is still in the early stages, and we have no implementation experience whatsoever with such an `exception_list`. Because `exception_list` is an exception type, extending its interface in the future would radically change how it can be implemented (today it can be implemented with a

`vector<exception_ptr>`; in the future it would need to be a persistent container of some type) and thus would likely require ABI-breaking backwards-incompatible changes.

The authors of [P0394r4] and this paper feel that the adoption of [P0394r4] is a step in the right direction. We believe the re-introduction of `exception_list` or inconsistencies in exception handling between different execution policies to the C++17 would decrease the quality of the standard in general and the parallel algorithms library in particular.

[P0394r4] does not preclude the addition of a better exception handling mechanism in the future. In the context of the executors proposal ([P0443r0]), there is an implicit "default" executor that is currently used with all execution policies. One backwards-compatible extension approach would be to make the `terminate()`-on-uncaught-exception behavior a property of this implicit "default" executor. Alternatively, an executor agnostic approach could be taken by introducing new execution policies with a different exception handling mechanism (e.g. `seq_except`, `par_except` and `par_unseq_except`).

However, since the 2016 Oulu meeting, a number of individuals (including the authors) have suggested changes and improvements to [P0394r4] via US national ballot comments. This paper addresses those suggestions and proposes a few possible resolutions for those comments that will ensure that `exception_list` and other parallel exception handling mechanisms can be added in the future with the introduction of executors.

§ 2. Feedback on P0394r4

After Oulu, Alisdair sent the following mail:

The key take-away I got from the SG1 session is that we might extend the parallel policies in the future, with throwing policies, when we better understand the domain. The main problem with this is that will mean rewriting each algorithm for each new policy, essentially for only error handling.

I think we could add customizable error handling to the current scheme by making the policy type incorporate the named tag, and a static `fail()` function - where all of the current policies `fail()` by simply calling `terminate()`. This would allow us to compose into the future with a policy that instead `failed` by throwing an exception, etc.

The main problem with this (other than being late) is that we don't have any context of the failure captured with a simple `fail()` call taking no arguments. I still think that would be better than leaving no customization point to embed in the existing algorithm implementations.

Bryce submitted a US national ballot comment about this issue:

The current wording does not leave the door open for executors (a feature under development by SG1) to modify the exception-handling behaviour of parallel algorithms in the future without breaking backwards compatibility.

The proposed resolution for this comment is to add a customization point in the `execution` namespace instead of adding a method to the execution policy types.

However, the authors now concur that a `fail()` method is a better approach, although a non-static member may be a better choice than a static member. While the execution policies are currently just tag types, the executor paper ([P0443r0]) proposes adding an interface to execution policies to allow them to compose with executors. Since execution policies are likely to have an interface in the future anyways, adding a customization point would cause inconsistent. Such a `fail()` function would likely need specific wording (similar to `terminate()`) allowing making it implementation-defined whether or not the stack is unwound before `fail()` is called.

Another ballot comment was submitted suggesting that the `terminate()`-on-uncaught-exception behavior might become a pitfall:

Calling `terminate()` when an element access function exits via an uncaught exception effectively disables the normal means of C++ error handling and propagation when using the parallel algorithms. This will be both confusing to users and a common source of bugs. Furthermore, by defining this behavior we are essentially preventing further solutions to this problem.

The authors agree. This behavior is not ideal and we must make sure that the changes in [\[P0394r4\]](#) do not prevent future improvements.

The comment suggests the following possible solutions:

#1. Make it undefined behavior when an element access function exits via an uncaught exception. This will allow for a future solution to this problem that is backwards compatible.

The authors believe this approach would work. In fact, when [\[P0394r4\]](#) was written, the general consensus among us was that undefined behavior was probably a better approach, but the caveats and implications of calling `terminate()` were better understood. We are open to seeing if there is consensus on the committee for changing to undefined behavior here.

This option would allow an implementation to implement the exception handling behavior described in the Parallelism TS v1 and throw an `exception_list`. Said `exception_list` might not be compatible with a future standardized `exception_list`, but we do not believe this is a substantial concern. Based on conversations with implementers, we think most implementations would not take this approach and would just call `terminate()` if we switched to undefined behavior.

#2. When an element access function exits via an uncaught exception, throw a `std::exception_list` which represents a collection of exceptions that were thrown in parallel.

We still believe this is not a viable option because of the issues with `exception_list` described in [§1 Background](#) and [\[P0394r4\]](#).

#3. When an element access function exits via an uncaught exception, throw an unspecified `std::exception`.

This approach will not work because it would re-introduce inconsistencies between different execution policies. The potential for exceptions - any exceptions, not just `exception_lists` - escaping from element access functions introduces control flow divergence. This divergence prevents significant hurdles on vector hardware. On such platforms that also support shared libraries, it is very difficult for a compiler to prove that this divergence is not possible even if none of the element access functions could possibly throw. The implementation must assume that any external function in a shared library which is not marked `noexcept` might cause control flow divergence due to different exit points, even if the external function actually does not throw an exception. We believe that inconsistencies in exception handling behavior between the different kinds of execution policies (`seq`, `par` and `par_unseq`) are undesirable as they will introduce pitfalls and force users to learn caveats.

#4. Rename the parallel algorithms to clarify that exception throwing code will result in a call to `std::terminate`. For example `std::execution::parallel_policy` would be renamed to `std::execution::parallel_policy_noexcept` and `std::execution::par` would be renamed to `std::execution::par_noexcept`.

The authors believe this approach would decrease the usability and elegance of the parallel algorithms interface. Users of the parallel algorithms library will be writing the execution policy tags (`seq`, `par` and `par_unseq`) frequently, so shorter identifiers are desirable. Renaming the execution policy types would be acceptable, but confusing if we later add `seq_except`, `par_except` and `par_unseq_except`.

Other comments suggest that `exception_list` should be re-introduced. We agree that it would be nice to have, but we disagree that it should be in C++17 for the reasons outlined in [§1 Background](#) and [\[P0394r4\]](#).

3. Proposed Resolutions

§

We suggest that the committee select one of the two following resolutions to the aforementioned ballot comments:

1. Make it undefined behavior for an element access function to exit via an uncaught exception.
2. Add a `fail()` method which calls `terminate()` to the three execution policies and have the `fail()` method be called. SG1 and LEWG should decide if a static or non-static method would be better.
3. Do #2 and also make it implementation-defined whether or not the stack is unwound before `fail()` is called.

§ 4. Acknowledgement

Thanks to:

- David Sankel for his invaluable and thoughtful feedback and interest in resolving this issue.
- JF Bastien for co-authoring [\[P0394r4\]](#) and providing feedback on this paper.

§ References

§ Informative References

[P0394r4]

JF Bastien, Bryce Adelstein Lelbach. [Hotel Parallelifornia: terminate\(\) for Parallel Algorithms Exception Handling](#). 23 June 2016. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0394r4.html>

[P0443r0]

Jared Hoberock, Michael Garland, Chris Kohlhoff, Chris Mysisen, Carter Edwards. [A Unified Executors Proposal for C++](#). 17 October 2016. URL: <http://wg21.link/P0443>